SUDOKU AS A

CONSTRAINT SATISFACTION PROBLEM

Classical Methods · AI-Assisted Solving · Live Code Walkthrough

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Institut supérieur de gestion de tunis

THE TEAM

Eya Ibn Abdelhedi

Amira Midani

Ghofrane Araar

WHAT WILL WE LEARN TODAY?

01 WHAT IS A CSP?

VARIABLES, DOMAINS, CONSTRAINTS WITH SIMPLE EVERYDAY EXAMPLES

02 THEORY

BACKTRACKING, HEURISTICS, ARC CONSISTENCY EXPLAINED WITH ANALOGIES

03 OUR CODE

WALK THROUGH THE ACTUAL PYTHON CODE WE WROTE, FUNCTION BY FUNCTION

04 BENCHMARKS

HOW FAST IS OUR SOLVER? COMPARE AGAINST GOOGLE'S SOLVER

05 COMPLEXITY

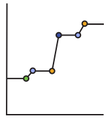
WHAT DOES $O(D^n)$ REALLY MEAN? (SPOILER: IT'S VERY, VERY BIG)

06 LIMITS & TAKEAWAYS

WHAT WE CAN'T DO YET AND WHERE TO GO NEXT

WHAT IS A CONSTRAINT SATISFACTION PROBLEM?

A CSP IS A PUZZLE WHERE YOU MUST FILL IN VALUES, BUT SOME COMBINATIONS ARE FORBIDDEN.



VARIABLE

A THING WE NEED TO DECIDE.
EXAMPLE: "WHAT DIGIT GOES IN ROW 2, COLUMN 3?"

{x} DOMAIN

THE LIST OF VALUES THAT VARIABLE IS ALLOWED TO TAKE.
EXAMPLE: {1, 2, 3, 4, 5, 6, 7, 8, 9}

CONSTRAINT

A RULE THAT SAYS WHICH COMBINATIONS ARE NOT ALLOWED.
EXAMPLE: "NO TWO CELLS IN THE SAME ROW CAN HAVE THE SAME DIGIT."

THE GOAL: FIND VALUES FOR ALL VARIABLES THAT BREAK NONE OF THE CONSTRAINTS.

CSPS APPEAR EVERYWHERE : THREE EXAMPLES

MAP COLOURING

Variables: each country. Domain: {red, green, blue, yellow}.
Constraint: neighbouring countries must have different colours.
You've probably solved this as a kid , it's a CSP!

SUDOKU

Variables: 81 cells. Domain: {1–9} for empty cells.
Constraint: every row, column, and 3×3 box must contain all 9 digits exactly once.

This is what we're solving!

EXAM SCHEDULING

Variables: each exam. Domain: available time slots.
Constraint: two exams can't happen at the same time if students attend both.

Your university does this every semester!

WHY SUDOKU? (THE SEARCH SPACE IS ENORMOUS)

- A 9×9 Sudoku has 81 cells. Each empty cell could hold a digit from 1 to 9.
- If we tried every possible combination blindly, we'd have up to 9^{81} options , that's roughly 2×10^{77}
- For comparison: the number of atoms in the observable universe is ~10^{80}
- Even at a billion checks per second, brute force would take longer than the age of the universe
- BUT,with the right techniques, most 9×9 puzzles solve in under 0.1 seconds on a normal laptop!

9^{81}

≈ 2×10^{77} possible grids

0.08 S

time with our heuristic solver

$10^{77}\times$

speed-up from smart algorithms

SUDOKU AS A CSP : THE VARIABLES

A CSP IS A PUZZLE WHERE YOU MUST FILL IN VALUES, BUT SOME COMBINATIONS ARE FORBIDDEN.

1	?	?	4
?	4	1	?
?	1	4	?
4	?	?	1

4×4 example
(? = variable)

In a 4×4 Sudoku, each "?" is a variable we need to decide its value

- In a 9×9 Sudoku: 81 cells total, up to 81 variables
- If a cell already has a clue (a given digit), it is NOT a variable its value is fixed
- We name each variable by its position: (row, column). Example: (0, 2) = row 0, column 2
- Standard easy 9×9 has 30 clues → 51 variables to solve

SUDOKU AS A CSP : THE DOMAINS

A CSP IS A PUZZLE WHERE YOU MUST FILL IN VALUES, BUT SOME COMBINATIONS ARE FORBIDDEN.

Empty cell (no clue)

Domain = {1, 2, 3, 4, 5, 6, 7, 8, 9}
All 9 digits are possible at the start!

Clue cell (digit given)

Domain = {5} (or whatever the clue is)
Only one possible value it's already decided.

After some propagation

Domain = {2, 7} (for example)
Half the options were eliminated by the rules!

- As we make assignments and apply rules, domains SHRINK
- If a domain reaches size 1 → we know exactly what goes in that cell
- If a domain reaches size 0 → we've made a mistake somewhere and must backtrack
- Shrinking domains early = fewer things to try = faster solving

SUDOKU AS A CSP : THE CONSTRAINTS

A CSP IS A PUZZLE WHERE YOU MUST FILL IN VALUES, BUT SOME COMBINATIONS ARE FORBIDDEN.

ROW RULE

Every row must contain the digits 1–9 each exactly once. No repeats!

COLUMN RULE

Every column must contain the digits 1–9 each exactly once.

BOX RULE

Every 3×3 box must contain the digits 1–9 each exactly once.

What we call it	How many?	In maths
Row constraints	9 (one per row)	AllDifferent({row cells})
Column constraint	9 (one per column)	AllDifferent({col cells})
Box constraint	9 (one per box)	AllDifferent({box cells})
Total	27 constraints	=972 “not equal” pairs

NAÏVE BACKTRACKING : TRY EVERYTHING

BACKTRACKING = "TRY A VALUE, KEEP GOING, IF STUCK → UNDO AND TRY THE NEXT VALUE."

1 PICK ANY EMPTY CELL	Start with cell(0,0). it has domain {1...9}
2 TRY VALUE 1	Write 1 into cell (0,0). Does it break any rule? Check the row, col, box
3 IF OK MOVE ON	Pick the next empty cell and try its values recursively
4 IF STUCK (NO VALUE WORKS)	We made a wrong choice earlier. Go back (backtrack) and try the next value
5 REPEAT UNTIL SOLVED	Eventually either we fill every cell(solved) or exhaust all options(no solution)

WHY NAÏVE BACKTRACKING IS WAY TOO SLOW

- WITH NO SMART CHOICES, WE PICK CELLS IN ORDER AND TRY DIGITS 1, 2, 3 … ONE BY ONE
- FOR A 9×9 PUZZLE WITH 51 EMPTY CELLS: WORST CASE = $9^{51} \approx 9 \times 10^{48}$ ATTEMPTS
- THAT'S 90 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 000 ATTEMPTS
- RUNNING AT 1 BILLION CHECKS PER SECOND: IT WOULD TAKE 10^{30} YEARS
- THE UNIVERSE IS ONLY 14 BILLION YEARS OLD…

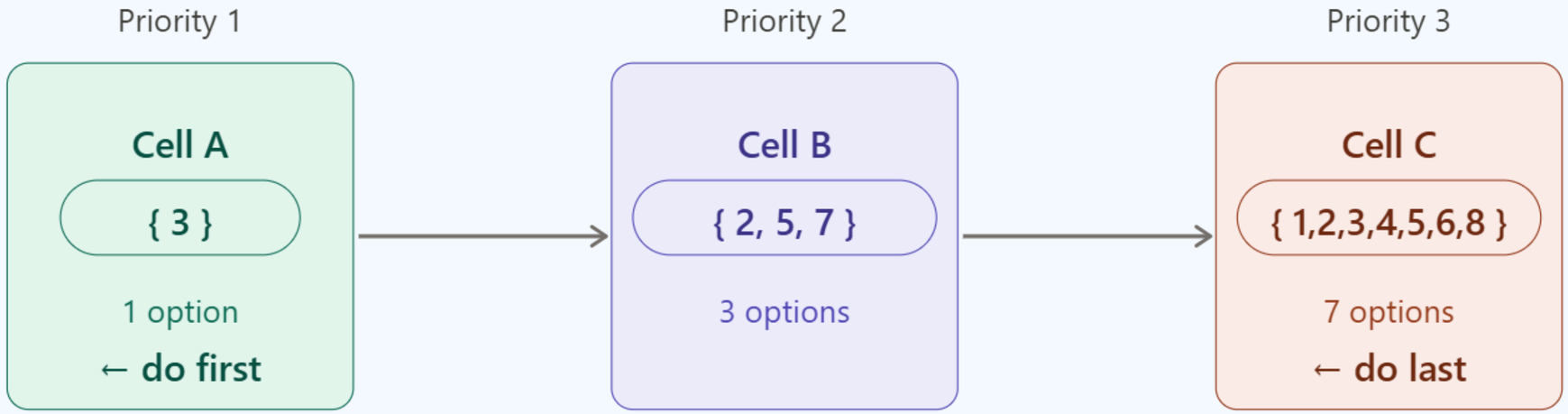
Small example: 3 variables, each with domain {1,2,3}

Total combination=3^3 =27 | for 9 variables: 9^9 =387 million | for 81 variables: 9^{81}=10

Every extra variable multiplies the search space. This is called exponential growth.

HEURISTIC 1 : MRV (MINIMUM REMAINING VALUES)

MRV = "ALWAYS FILL IN THE CELL THAT HAS THE FEWEST LEGAL OPTIONS LEFT."



Why? — the fail-first strategy

Zero-risk cell
Domain size 1 = only one possible value

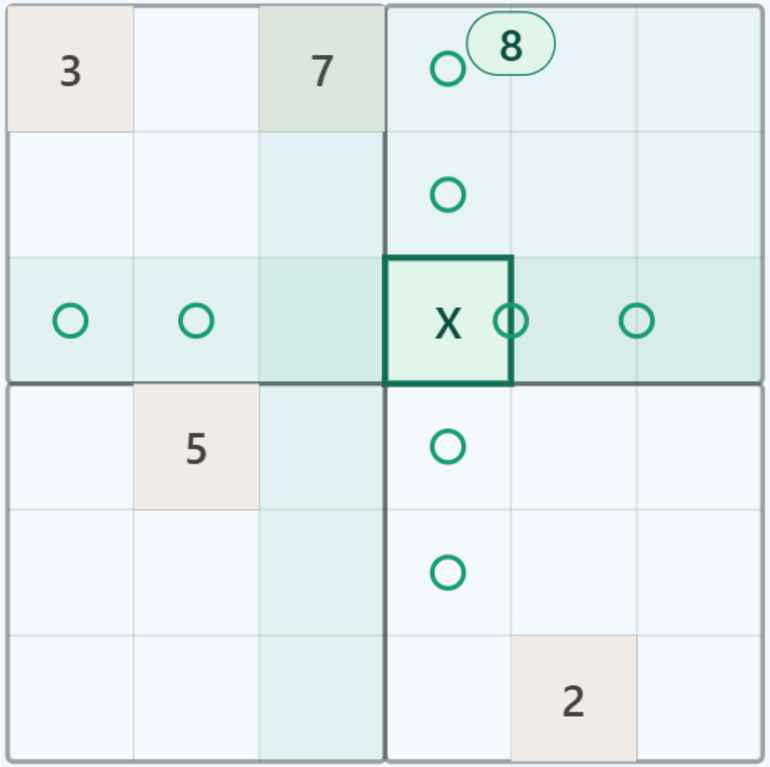
Shrinks neighbours
Placing a digit removes it from 20 peers' domains

Catches failures early
Tight cells fail fast — backtrack before digging deep

MRV rule
Always pick the cell with the smallest domain → sort A → B → C

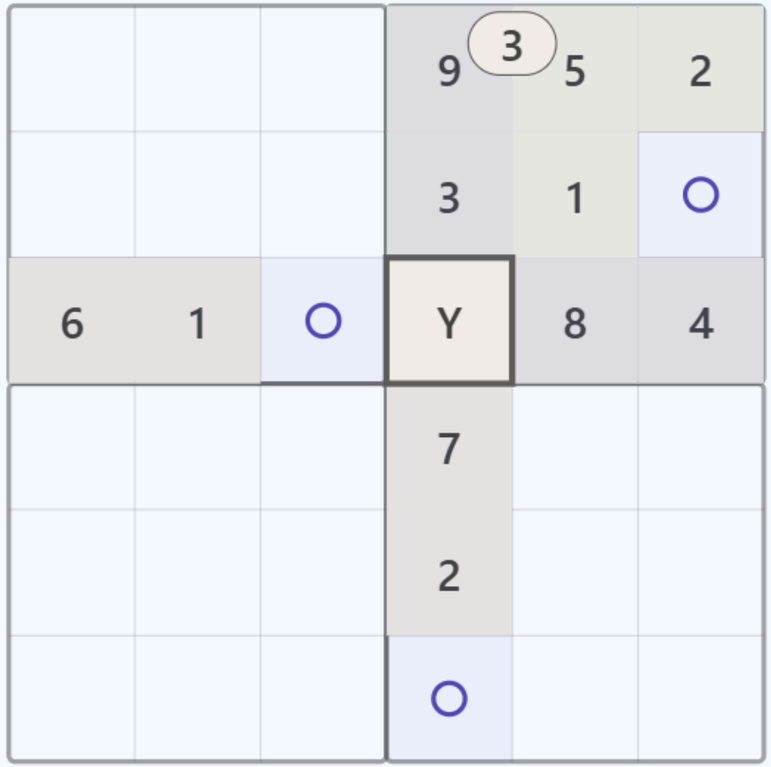
HEURISTIC 2 : DEGREE (BREAK MRV TIES)

WHEN TWO CELLS TIE ON MRV, PICK THE ONE CONNECTED TO THE MOST OTHER UNASSIGNED CELLS.



Cell X

8 unassigned peers
mostly empty grid



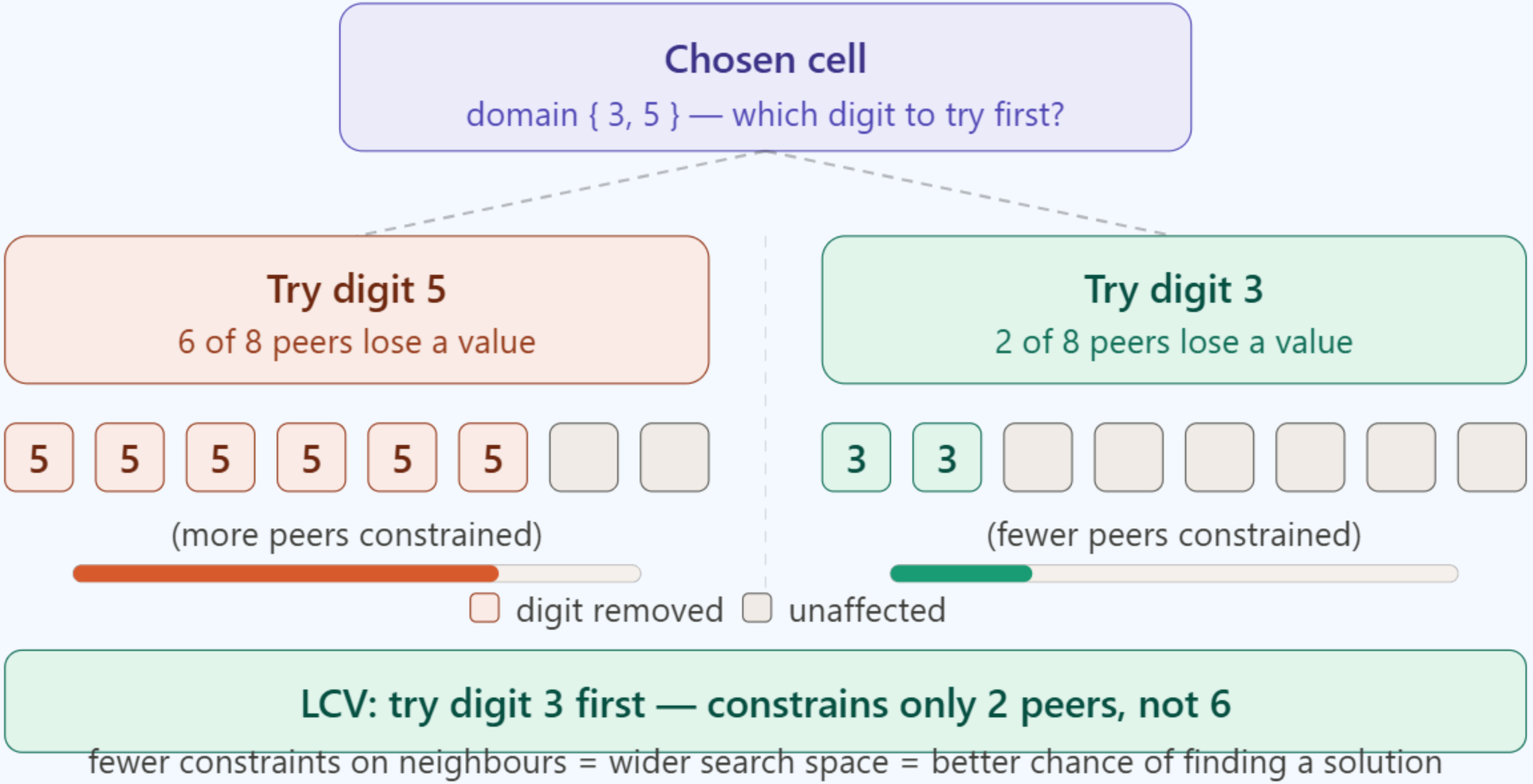
Cell Y

3 unassigned peers
mostly filled grid

cell to assign unassigned peer already filled

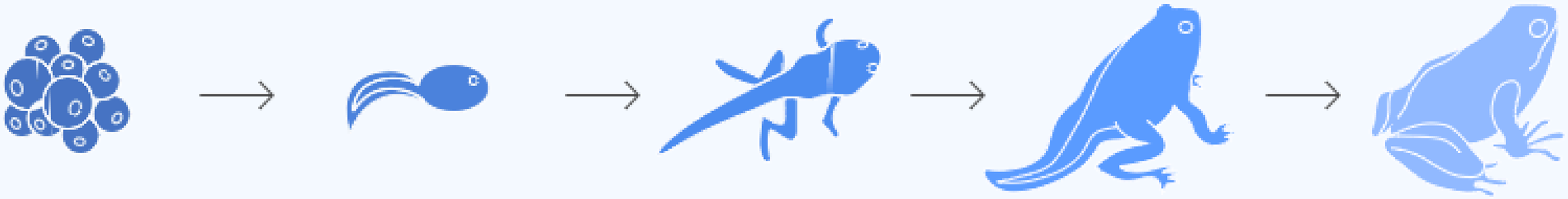
HEURISTIC 3 : LCV (LEAST CONSTRAINING VALUE)

LCV = "WHEN CHOOSING WHICH DIGIT TO TRY FIRST, PICK THE ONE THAT RULES OUT THE FEWEST OPTIONS FOR YOUR NEIGHBOURS."



FORWARD CHECKING — PRUNE DEAD ENDS EARLY

FORWARD CHECKING = "AS SOON AS YOU ASSIGN A VALUE, IMMEDIATELY REMOVE IT FROM ALL NEIGHBOURS' DOMAINS."

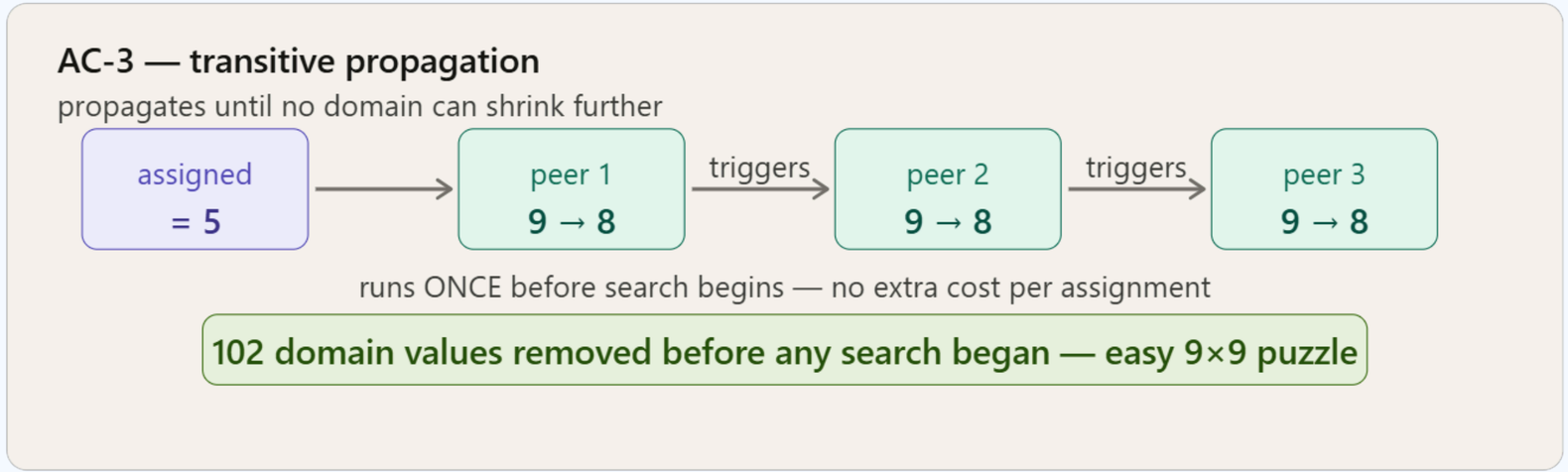


Initial Domains	Remove Value	Check for Empty	Backtrack	Reduced Domains
All cells have possible values	Eliminate assigned value from neighbours	Detect if any neighbour's domain is empty	Undo assignment if domain is empty	Domains are smalle and more constrained

AC-3 :A STRONGER CLEANUP BEFORE WE EVEN SEARCH

ARC CONSISTENCY: FOR EVERY VALUE V IN CELL X, THERE MUST BE AT LEAST ONE COMPATIBLE VALUE IN EVERY NEIGHBOUR Y.

IF NO COMPATIBLE VALUE EXISTS → REMOVE V FROM X'S DOMAIN BEFORE WE EVEN START SEARCHING.



Forward checking · 1 hop · cheap per step

AC-3 · paid once upfront · fully transitive

OUR CODE COMPLETE WALKTHROUGH

01 SUDOKUCSP CLASS

GRID, DOMAINS, PEERS, PROPAGATE

02 AC-3

ARC CONSISTENCY PRE-PROCESSING

03 HEURISTICS

MRV + DEGREE + LCV

04 BACKTRACKING

RECURSIVE SEARCH + SOLVE()

05 PUZZLES

4×4, 9×9 EASY/HARD, 16×16

06 BENCHMARKS

SCALABILITY TABLE RUNNER

07 OR-TOOLS

COMPARISON SOLVER

08 ASSISTANT

INTERACTIVE PUZZLE ENTRY

09 MAIN

CLI ENTRY POINT + DEMO

IMPORTS & TYPE ANNOTATIONS

```
2  import copy, time, math, argparse, sys
3    from collections import deque
4    from typing import Dict, List, Optional, Set, Tuple
5
```

1 COPY

USED FOR DEEPCOPY() IN BACKTRACKING SAVES A FULL SNAPSHOT OF DOMAINS AND GRID BEFORE EACH ASSIGNMENT SO WE CAN UNDO PERFECTLY.

2 TIME

TIME.PERF_COUNTER() GIVES NANOSECOND PRECISION. USED TO MEASURE HOW LONG EACH SOLVE TAKES IN THE BENCHMARK

3 MATH

MATH.ISQRT() COMPUTES THE INTEGER SQUARE ROOT — USED TO GET THE BOX SIZE FROM THE GRID SIZE (√9=3, √16=4).

4 ARGPARSE

PARSES --TRACE, --BENCH, --ASSISTANT FLAGS FROM THE COMMAND LINE SO THE FILE HAS MULTIPLE RUN MODES.

5 DEQUE

DOUBLE-ENDED QUEUE FROM COLLECTIONS. USED AS THE AC-3 ARC QUEUE — O(1) APPEND AND POPLEFT, MUCH FASTER THAN A LIST.

6 TYPING

DICT, LIST, OPTIONAL, SET, TUPLE FOR TYPE HINTS
MAKES THE CODE SELF-DOCUMENTING AND IDE-FRIENDLY.

SUDOKUCSP — __INIT__ : CLASS SETUP

```python
lass SudokuCSP:
    def __init__(self, size: int = 9):
        self.size     = size
        self.box_size = int(math.isqrt(size))
        if self.box_size * self.box_size != size:
            raise ValueError(f"Grid size must be a perfect square; got {size}")


        self.grid: List[List[int]] = [[0]*size for _ in range(size)]

        self.domains: List[List[set]] = [
            [set(range(1, size+1)) for _ in range(size)]
            for _ in range(size)
        ]

        self._peer_cache: Dict[Tuple, List[Tuple]] = {}

        self.nodes_visited: int = 0
        self.backtracks:    int = 0
```

1

SIZE & BOX_SIZE

SIZE=9 FOR STANDARD SUDOKU. BOX_SIZE=INT(√9)=3. THE VALUE ERROR GUARD ENSURES WE ONLY ACCEPT 4, 9, 16 , ANY NON PERFECT SQUARE CRASHES EARLY WITH A CLEAR MESSAGE.

2

SELF.GRID

TIME.PERF_COUNTER() GIVES NANOSECOND PRECISION. USED TO MEASURE HOW LONG EACH SOLVE TAKES IN THE BENCHMARK

3

SELF.DOMAINS

MATH.ISQRT() COMPUTES THE INTEGER SQUARE ROOT — USED TO GET THE BOX SIZE FROM THE GRID SIZE (√9=3, √16=4).

4

_PEER_CACHE

COMPUTING PEERS IS SLOW IF DONE REPEATEDLY. WE COMPUTE EACH CELL'S 20 NEIGHBOURS ONCE AND STORE THE RESULT. ON A 9×9 THAT'S 81 LOOKUPS SAVED PER BACKTRACK LEVEL.

5

NODES_VISITED & BACKTRACKS

STATISTICS COUNTERS. INCREMENTED DURING SEARCH. PRINTED IN THE BENCHMARK OUTPUT TO COMPARE NAIVE VS MRV.

SUDOKUCSP — PEERS() : FIND ALL 20 NEIGHBOURS

```python
def peers(self, r: int, c: int) -> List[Tuple[int,int]]:
    key = (r, c)
    if key not in self._peer_cache:
        ps: Set[Tuple] = set()
        for cc in range(self.size):
            if cc != c: ps.add((r, cc))
        for rr in range(self.size):
            if rr != r: ps.add((rr, c))
        br = (r // self.box_size) * self.box_size
        bc = (c // self.box_size) * self.box_size
        for rr in range(br, br + self.box_size):
            for cc in range(bc, bc + self.box_size):
                if (rr, cc) != (r, c):
                    ps.add((rr, cc))
        self._peer_cache[key] = list(ps)
    return self._peer_cache[key]
```

1

WHY 20 PEERS?

A 9×9 CELL SHARES ITS ROW (8 OTHERS) + COLUMN (8 OTHERS) + BOX (8 OTHERS) BUT ROW∩BOX AND COL∩BOX OVERLAP. THE SET DEDUPLICATES, GIVING EXACTLY 20 UNIQUE PEERS PER CELL.

2

BOX ORIGIN FORMULA

BR = (R // BOX_SIZE) * BOX_SIZE FINDS THE FIRST ROW OF THIS CELL'S BOX. EXAMPLE: ROW 7 → 7//3=2, 2*3=6. SO THE BOX STARTS AT ROW 6. SAME LOGIC FOR COLUMNS.

3

USING A SET FIRST

WE COLLECT INTO PS (A PYTHON SET) TO AVOID DUPLICATES FROM CORNER CELLS THAT APPEAR IN BOTH THE ROW AND THE BOX. THEN CONVERT TO LIST FOR STORAGE.

4

CACHING IN _PEER_CACHE

EVERY CALL AFTER THE FIRST IS JUST A DICT LOOKUP — O(1). WITHOUT CACHING, EVERY _PROPAGATE CALL RECOMPUTES PEERS. ON A HARD PUZZLE WITH 1000 BACKTRACKS THAT'S MILLIONS OF WASTED CALCULATIONS.

5

CONSTRAINT ENCODING

THESE 20 PEERS REPRESENT ALL THE ≠ CONSTRAINTS FOR A CELL. THE CSP DEFINITION SAYS ALLDIFFERENT(ROW) + ALLDIFFERENT(COL) + ALLDIFFERENT(BOX). PEERS() UNIFIES ALL THREE INTO ONE LIST.

SUDOKUCSP — LOAD() : INITIALISE FROM A PUZZLE GRID

```python
def load(self, puzzle: List[List[int]]) -> bool:

    for r in range(self.size):
        for c in range(self.size):
            v = puzzle[r][c]
            if v != 0:
                self.grid[r][c]    = v
                self.domains[r][c] = {v}

    for r in range(self.size):
        for c in range(self.size):
            if self.grid[r][c] != 0:
                if not self._propagate(r, c, self.grid[r][c]):
                    return False
    return True
```

1

TWO-PASS DESIGN

PASS 1 COPIES ALL CLUES. PASS 2 PROPAGATES THEM. KEEPING THEM SEPARATE AVOIDS A BUG: IF WE PROPAGATED DURING PASS 1, SOME PEERS MIGHT NOT HAVE THEIR CLUE VALUE SET YET.

2

SELF.DOMAINS[R][C] = {V}

WHEN A CELL HAS A CLUE, WE COLLAPSE ITS DOMAIN FROM {1..9} DOWN TO JUST {V}. THIS MEANS MRV WILL ALWAYS PICK FORCED CELLS FIRST , THEY HAVE DOMAIN SIZE 1.

3

CALLING _PROPAGATE PER CLUE

EACH CLUE REMOVES ITS VALUE FROM 20 PEER DOMAINS. A 9×9 EASY PUZZLE HAS 30 CLUES, SO LOAD() TRIGGERS 30 PROPAGATIONS BEFORE SEARCH EVEN STARTS. THIS IS FREE PRUNING.

4

RETURNS FALSE FOR BROKEN PUZZLES

IF TWO CLUES IN THE SAME ROW SHARE THE SAME DIGIT, PROPAGATION WILL WIPE A NEIGHBOUR'S DOMAIN TO EMPTY. LOAD() RETURNS FALSE AND SOLVE() ABORTS IMMEDIATELY, NO WASTED SEARCH.

SUDOKUCSP — _PROPAGATE() : FORWARD CHECKING

```python
def _propagate(self, r: int, c: int, value: int) -> bool:

    for pr, pc in self.peers(r, c):
        if self.grid[pr][pc] == 0:
            self.domains[pr][pc].discard(value)
            if not self.domains[pr][pc]:
                return False
    return True
```

1

WHAT IT DOES

AFTER PLACING VALUE AT (R,C), EVERY PEER THAT STILL HAS VALUE IN ITS CANDIDATE LIST MUST LOSE IT. THAT DIGIT IS NOW TAKEN, PEERS CAN'T USE IT ANYMORE.

2

.DISCARD() NOT .REMOVE()

DISCARD() SILENTLY DOES NOTHING IF VALUE ISN'T IN THE SET. REMOVE() RAISES A KEYERROR. WE USE DISCARD BECAUSE A PEER MIGHT ALREADY HAVE HAD VALUE PRUNED BY AN EARLIER PROPAGATION.

3

IF NOT SELF.DOMAINS[PR][PC]

AN EMPTY SET IN PYTHON IS FALSY. THIS CHECKS 'DID WE JUST REMOVE THE LAST LEGAL VALUE FROM THIS PEER?'

?' IF YES, THERE IS NO VALID DIGIT FOR THAT CELL, WE HAVE A CONTRADICTION. RETURN FALSE IMMEDIATELY.

4

ONLY CHECKS UNASSIGNED PEERS

IF GRID[PR][PC] != 0, THE CELL IS ALREADY FILLED, ITS DOMAIN IS IRRELEVANT. SKIPPING IT SAVES WORK AND AVOIDS ACCIDENTALLY PRUNING A DOMAIN WE NO LONGER NEED.

5

CALLED AFTER EVERY ASSIGNMENT

_PROPAGATE RUNS INSIDE _BACKTRACK FOR EVERY DIGIT WE TRY. ON A HARD PUZZLE WITH 1081 NODES VISITED, THIS RUNS AT LEAST 1081 TIMES. IT MUST BE FAST, AND IT IS, SINCE PEERS() IS CACHED.

SUDOKUCSP — HELPER METHODS

```python
def unassigned(self) -> List[Tuple[int,int]]:
    return [(r, c) for r in range(self.size)
            for c in range(self.size) if self.grid[r][c] == 0]

def is_solved(self) -> bool:
    return all(self.grid[r][c] != 0
               for r in range(self.size) for c in range(self.size))


def print_grid(self):
    b = self.box_size
    w = len(str(self.size)) + 1
    sep = ("—" * (self.size * w + b - 1))
    for r in range(self.size):
        if r % b == 0 and r != 0:
            print("  " + sep)
        row = "  "
        for c in range(self.size):
            if c % b == 0 and c != 0:
                row += "|"
            v = self.grid[r][c]
            row += (f"{v:>{w}}" if v else " " + "."*(w-1))
        print(row)
    print()
```

1 **UNASSIGNED**

LIST COMPREHENSION OVER ALL 81 CELLS. RETURNS ONLY THOSE WITH GRID VALUE 0. CALLED BY MRV_SELECT() AND NAIVE_SELECT() TO GET THE CANDIDATE LIST OF CELLS TO FILL NEXT.

2 **IS_SOLVED()**

ALL() SHORT-CIRCUITS: STOPS AS SOON AS IT FINDS A 0. THE BASE CASE OF _BACKTRACK: IF IS_SOLVED() RETURNS TRUE, WE RETURN TRUE ALL THE WAY UP THE CALL STACK.

3 **PRINT_GRID() — CELL WIDTH**

W = LEN(STR(SIZE))+1 GIVES WIDTH 2 FOR 9×9 (SINGLE DIGIT) AND WIDTH 3 FOR 16×16 (TWO-DIGIT NUMBERS). THIS MAKES THE GRID PRINT CLEANLY FOR ANY SIZE.

4 **BOX SEPARATOR LOGIC**

R % B == 0 AND R != 0 PRINTS A HORIZONTAL LINE OF CHARACTERS BETWEEN EVERY GROUP OF 3 ROWS. SAME LOGIC FOR | VERTICAL SEPARATORS BETWEEN BOX COLUMNS.

5 **CONSTRAINT ENCODING**

AN UNASSIGNED CELL PRINTS A DOT (.) INSTEAD OF 0 TO MATCH STANDARD SUDOKU NOTATION. THE F-STRING HANDLES DIGIT ALIGNMENT SO COLUMNS STAY STRAIGHT.

AC-3 — QUEUE SETUP : BUILDING ALL DIRECTED ARCS

```python
def ac3(csp: SudokuCSP) -> Tuple[bool, int]:
    queue: deque = deque()
    for r in range(csp.size):
        for c in range(csp.size):
            for pr, pc in csp.peers(r, c):
                queue.append(((r,c),(pr,pc)))

    removed = 0
```

1 WHY DIRECTED ARCS?

AC-3 CHECKS: FOR EVERY VALUE IN DOMAIN(XI), DOES DOMAIN(XJ) HAVE AT LEAST ONE COMPATIBLE VALUE? DIRECTION MATTERS , ARC (XI→XJ) TESTS XI'S DOMAIN AGAINST XJ'S. (XJ→XI) DOES THE REVERSE.

2 HOW MANY ARCS?

9×9 SUDOKU: 972 ALLDIFFERENT PAIRS × 2 DIRECTIONS = 1,944 DIRECTED ARCS IN THE INITIAL QUEUE. THIS SOUNDS LIKE A LOT BUT THE QUEUE DRAINS QUICKLY FOR EASY PUZZLES.

3 USING DEQUE, NOT LIST

IF GRID[PR][PC] != 0, THE CELL IS ALREADY FILLED, ITS DOMAIN IS IRRELEVANT. SKIPPING IT SAVES WORK AND AVOIDS ACCIDENTALLY PRUNING A DOMAIN WE NO LONGER NEED.

4 REMOVED COUNTER

WE COUNT HOW MANY VALUES AC-3 REMOVES FROM DOMAINS. FOR THE EASY 9×9, THIS PRINTS AS 'AC3-=102' IN THE BENCHMARK. IT MEASURES HOW MUCH WORK AC-3 SAVED THE SEARCH.

AC-3 — MAIN LOOP : REVISE ARCS UNTIL NOTHING CHANGES

```
    while queue
        (xi, xj) = queue.popleft()
        ri, ci = xi; rj, cj = xj

        new = {v for v in csp.domains[ri][ci]
               if any(v != w for w in csp.domains[rj][cj])}
        delta = len(csp.domains[ri][ci]) - len(new)
        if delta:
            removed += delta
            csp.domains[ri][ci] = new
            if not csp.domains[ri][ci]:
                return False, removed

            for pk, pl in csp.peers(ri, ci):
                if (pk,pl) != (rj,cj):
                    queue.append(((pk,pl), (ri,ci)))
    return True, removed
```

1

THE REVISE CHECK

'NEW' KEEPS ONLY VALUES V WHERE AT LEAST ONE W IN DOMAIN(XJ) SATISFIES W≠V. SINCE ALL CONSTRAINTS ARE ≠, A VALUE V IN XI IS DEAD ONLY IF DOMAIN(XJ)={V} — XJ IS ALREADY FIXED TO THAT SAME VALUE.

2

DELTA: DID ANYTHING CHANGE?

IF DOMAIN(XI) DIDN'T SHRINK (DELTA==0), THERE'S NOTHING NEW TO PROPAGATE. WE ONLY RE-QUEUE ARCS WHEN XI'S DOMAIN ACTUALLY CHANGED ,THIS KEEPS THE QUEUE SMALL.

3

EMPTY DOMAIN = CONTRADICTION

IF WE REMOVE THE LAST VALUE FROM XI'S DOMAIN, THE PUZZLE HAS NO SOLUTION FROM THIS STATE. AC3() RETURNS (FALSE, REMOVED) IMMEDIATELY. IN SOLVE(), THIS STOPS THE SEARCH BEFORE IT EVEN STARTS.

4

RE-QUEUE NEIGHBOURS OF XI

WHEN XI'S DOMAIN SHRINKS, CELLS THAT HAD XI AS A PEER MIGHT NOW HAVE VALUES THAT ONLY XI COULD SATISFY. WE ADD ARCS (XK→XI) TO RE-CHECK THEM. THIS IS WHAT MAKES AC-3 'TRANSITIVE'.

5

CONSTRAINT ENCODING

AN UNASSIGNED CELL PRINTS A DOT (.) INSTEAD OF 0 TO MATCH STANDARD SUDOKU NOTATION. THE F-STRING HANDLES DIGIT ALIGNMENT SO COLUMNS STAY STRAIGHT.

MRV_SELECT() : CHOOSE THE BEST CELL TO FILL NEXT

```python
def mrv_select(csp: SudokuCSP) -> Optional[Tuple[int,int]]:

    cells = csp.unassigned()
    if not cells:
        return None
    min_d = min(len(csp.domains[r][c]) for r,c in cells)
    candidates = [(r,c) for r,c in cells if len(csp.domains[r][c]) == min_d]
    if len(candidates) == 1:
        return candidates[0]

    assigned = {(r,c) for r in range(csp.size)
                for c in range(csp.size) if csp.grid[r][c] != 0}
    def degree(cell):
        r, c = cell
        return sum(1 for p in csp.peers(r,c) if p not in assigned)
    return max(candidates, key=degree)
```

1

WHY FAIL-FIRST?

CELLS WITH FEWER OPTIONS ARE MORE LIKELY TO CAUSE A CONTRADICTION. IF WE ASSIGN THEM EARLY, FAILURES ARE DETECTED SOONER. ON EASY 9×9: MRV REDUCES NODES FROM 334 TO 52.

2

MIN() OVER ALL CELLS

ONE PASS OVER ALL UNASSIGNED CELLS TO FIND THE MINIMUM DOMAIN SIZE. THEN A SECOND PASS TO COLLECT ALL CELLS THAT TIE AT THAT SIZE. TWO O(N) PASSES, NOT A SORT.

3

EARLY RETURN FOR NO TIE

IF ONLY ONE CELL HAS THE MINIMUM DOMAIN SIZE, RETURN IT IMMEDIATELY. SKIPS THE EXPENSIVE DEGREE COMPUTATION FOR MOST CALLS DURING EASY PUZZLE SOLVING.

4

DEGREE TIE-BREAK LOGIC

AMONG TIED CELLS, PICK THE ONE WITH THE MOST UNASSIGNED NEIGHBOURS. ASSIGNING THAT CELL WILL TRIGGER PROPAGATION TO MORE PEERS, PRUNING THE SEARCH SPACE MORE AGGRESSIVELY.

5

MAX(..., KEY=DEGREE)

MAX() WITH A KEY FUNCTION AVOIDS BUILDING A SORTED LIST, IT DOES ONE PASS AND RETURNS THE BEST ELEMENT. DEGREE() IS COMPUTED ONLY FOR TIED CANDIDATES.

NAIVE_SELECT() + LCV_ORDER() : BASELINE AND VALUE ORDERING

```python
def naive_select(csp: SudokuCSP) -> Optional[Tuple[int,int]]:
    """Baseline: first unassigned cell in reading order."""
    for r in range(csp.size):
        for c in range(csp.size):
            if csp.grid[r][c] == 0:
                return (r, c)
    return None


def lcv_order(csp: SudokuCSP, r: int, c: int) -> List[int]:

    def cost(v):
        return sum(1 for pr,pc in csp.peers(r,c)
                   if csp.grid[pr][pc] == 0 and v in csp.domains[pr][pc])
    return sorted(csp.domains[r][c], key=cost)
```

1

WHY KEEP NAIVE_SELECT?

IT'S THE BENCHMARK BASELINE. WHEN WE RUN 'NAIVE' CONFIG IN RUN_BENCHMARKS(), WE USE NAIVE_SELECT INSTEAD OF MRV_SELECT. WITHOUT IT WE COULDN'T MEASURE THE SPEED-UP THAT MRV PROVIDES.

2

NAIVE_SELECT IS O(N)

SCANS CELLS TOP-LEFT TO BOTTOM-RIGHT AND RETURNS THE FIRST EMPTY ONE. NO SORTING, NO DOMAIN INSPECTION. SIMPLE — AND SLOW, WHICH IS EXACTLY THE POINT OF THE COMPARISON.

3

LCV COST FUNCTION

COST(V) COUNTS HOW MANY UNASSIGNED PEERS CONTAIN V IN THEIR DOMAIN. A HIGH COST MEANS PLACING V HERE ELIMINATES MANY OPTIONS ELSEWHERE ,RISKY. LOW COST = SAFER FIRST TRY.

4

SORTED(..., KEY=COST)

SORTS THE ENTIRE DOMAIN OF (R,C) BY INCREASING COST. SO THE LEAST CONSTRAINING VALUE COMES FIRST. IF COST(3)=2 AND COST(5)=6, WE TRY 3 BEFORE 5.

5

LCV VS MRV INTERACTION

MRV PICKS WHICH CELL. LCV PICKS WHICH VALUE TO TRY FIRST. THEY WORK AS A TEAM: MRV MINIMISES THE RISK OF PICKING A CELL, LCV MINIMISES THE DAMAGE OF THE VALUE WE TRY.

_BACKTRACK() — PART 1 : SELECT CELL, SAVE STATE, TRACE

```python
def _backtrack(csp: SudokuCSP,
               use_mrv: bool, use_lcv: bool,
               trace: bool, depth: int = 0) -> bool:

    csp.nodes_visited += 1

    if csp.is_solved():
        return True

    cell = (mrv_select if use_mrv else naive_select)(csp)
    if cell is None:
        return False
    r, c = cell

    values = lcv_order(csp, r, c) if use_lcv else sorted(csp.domains[r][c])

    for v in values:
        if trace:
            pad = "  " * depth
            dom_size = len(csp.domains[r][c])
            print(f"{pad}→ cell({r+1},{c+1}) = {v}  "
                  f"[domain size was {dom_size}]")


        saved_domains = copy.deepcopy(csp.domains)
        saved_grid    = copy.deepcopy(csp.grid)
```

1

NODES_VISITED COUNTER

INCREMENTED AT THE VERY TOP OF EVERY CALL, BEFORE ANY WORK. THIS COUNTS EVERY STATE THE SOLVER VISITS. THE BENCHMARK PRINTS THIS TO SHOW HOW MANY CELLS MRV VS NAIVE ACTUALLY EVALUATED.

2

BASE CASE: IS_SOLVED()

IF THERE ARE NO MORE 0S IN THE GRID, WE'RE DONE. RETURN TRUE PROPAGATES UP THE ENTIRE CALL STACK, COMMITTING THE SOLUTION.

3

TERNARY SELECTOR

(MRV_SELECT IF USE_MRV ELSE NAIVE_SELECT)(CSP) , CHOOSES A FUNCTION OBJECT AT RUNTIME AND CALLS IT. ONE LINE REPLACES AN IF/ELSE BLOCK. THE BENCHMARK FLIPS USE_MRV TO COMPARE STRATEGIES.

4

TRACE MODE

WHEN --TRACE FLAG IS USED, PRINTS AN INDENTED LOG OF EVERY ASSIGNMENT DECISION. THE INDENTATION DEPTH SHOWS THE RECURSION LEVEL DEEPER = FURTHER DOWN THE SEARCH TREE.

5

DEEPCOPY: THE UNDO MECHANISM

COPY.DEEPCOPY CREATES A COMPLETELY INDEPENDENT COPY OF DOMAINS AND GRID. THIS IS EXPENSIVE (O(N×D) PER CALL) BUT ESSENTIAL , WITHOUT IT, BACKTRACKING WOULD CORRUPT THE STATE.

_BACKTRACK() — PART 2 : ASSIGN, CHECK, RECURSE, UNDO

```
    csp.grid[r][c]    = v
    csp.domains[r][c] = {v}


    ok = csp._propagate(r, c, v)
    if ok and _backtrack(csp, use_mrv, use_lcv, trace, depth+1):
        return True


    if trace:
        print(f"{'  '*depth}X backtrack ({r+1},{c+1})={v}")
    csp.backtracks += 1
    csp.domains = saved_domains
    csp.grid    = saved_grid

return False
```

1 **CSP.DOMAINS[R][C] = {V}**

AFTER ASSIGNING, WE COLLAPSE THE CELL'S OWN DOMAIN TO {V}. THIS ISN'T STRICTLY NEEDED FOR _PROPAGATE BUT IT PREVENTS MRV FROM EVER PICKING AN ALREADY-ASSIGNED CELL AGAIN.

2 **IF OK AND _BACKTRACK(...)**

SHORT-CIRCUIT: IF _PROPAGATE RETURNED FALSE (WIPE-OUT DETECTED), WE SKIP THE RECURSIVE CALL ENTIRELY. NO POINT SEARCHING DEEPER INTO AN ALREADY-BROKEN STATE.

3 **RETURN TRUE PROPAGATES UP**

WHEN _BACKTRACK RETURNS TRUE DEEP IN RECURSION, THE CALLING FRAME ALSO RETURNS TRUE IMMEDIATELY, SKIPPING ITS REMAINING VALUES. THE SOLUTION BUBBLES ALL THE WAY UP TO SOLVE().

4 **BACKTRACKS COUNTER**

ONLY INCREMENTED WHEN WE ACTUALLY UNDO AN ASSIGNMENT. ON EASY 9×9 WITH MRV: BACKTRACKS=0. ON AI ESCARGOT: BACKTRACKS=1100. THIS IS THE KEY BENCHMARK METRIC.

5 **RETURN FALSE**

IF NO VALUE IN THE DOMAIN WORKED, WE RETURN FALSE TO THE CALLER. THE CALLER THEN UNDOES ITS ASSIGNMENT AND TRIES THE NEXT VALUE. THIS IS THE BACKTRACKING CASCADE.

SOLVE() : THE PUBLIC ENTRY POINT

```python
def solve(puzzle: List[List[int]], size: int = 9,
          use_mrv: bool = True, use_lcv: bool = True,
          use_ac3: bool = True, trace: bool = False
          ) -> Tuple[Optional[SudokuCSP], float, int]:

    csp = SudokuCSP(size)
    ok  = csp.load(puzzle)
    if not ok:
        return None, 0.0, 0

    ac3_removed = 0
    if use_ac3:
        ok2, ac3_removed = ac3(csp)
        if not ok2:
            return None, 0.0, ac3_removed

    t0      = time.perf_counter()
    solved  = _backtrack(csp, use_mrv, use_lcv, trace)
    elapsed = time.perf_counter() - t0

    return (csp if solved else None), elapsed, ac3_removed
```

1

THREE-STEP PIPELINE

EVERY SOLVE CALL FOLLOWS THE SAME PIPELINE: LOAD (PLANT CLUES + FORWARD-CHECK), OPTIONALLY AC-3 (GLOBAL PRE-CLEAN), THEN BACKTRACKING SEARCH. FLAGS CONTROL WHICH STEPS RUN.

2

EARLY RETURN IF LOAD FAILS

IF THE PUZZLE ITSELF HAS TWO IDENTICAL CLUES IN THE SAME ROW, LOAD() RETURNS FALSE. SOLVE() IMMEDIATELY RETURNS NONE, NO WASTED SEARCH ON AN IMPOSSIBLE PUZZLE.

3

TIMER PLACEMENT

TIME.PERF_COUNTER() STARTS AFTER LOAD AND AC-3. THIS MEANS THE BENCHMARK MEASURES PURE SEARCH TIME, NOT I/O OR PRE-PROCESSING. THAT'S A DELIBERATE DESIGN CHOICE FOR FAIR COMPARISON.

4

RETURN TUPLE

RETURNS (CSP, ELAPSED, AC3_REMOVED). CALLERS UNPACK WITH R, T, AC3R = SOLVE(...). IF NO SOLUTION EXISTS, CSP=NONE. THE BENCHMARK CHECKS BOTH ELAPSED AND CSP.NODES_VISITED.

5

FLAGS FOR BENCHMARKING

USE_MRV=FALSE DISABLES MRV (USES NAIVE_SELECT). USE_AC3=FALSE SKIPS AC-3. THIS ONE FUNCTION COVERS ALL FOUR BENCHMARK CONFIGURATIONS JUST BY FLIPPING THREE BOOLEANS.

PUZZLE DEFINITIONS : FOUR TEST CASES

```
PUZZLE_4 = [
    [1, 0, 0, 4],
    [0, 4, 1, 0],
    [0, 1, 4, 0],
    [4, 0, 0, 1],
]


PUZZLE_9_EASY = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9],
]


PUZZLE_9_HARD = [
    [1, 0, 0, 0, 0, 7, 0, 9, 0],
    [0, 3, 0, 0, 2, 0, 0, 0, 8],
    [0, 0, 9, 6, 0, 0, 5, 0, 0],
    [0, 0, 5, 3, 0, 0, 9, 0, 0],
    [0, 1, 0, 0, 8, 0, 0, 0, 2],
    [6, 0, 0, 0, 0, 4, 0, 0, 0],
    [3, 0, 0, 0, 0, 0, 0, 1, 0],
```

```
PUZZLE_16 = [
    [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 11,  0,  0,  0,  0],
    [ 0,  5,  0,  0,  0,  3,  0,  0,  0,  0,  0,  0, 16,  0,  0,  0],
    [ 0,  0, 12,  0,  0,  0,  0,  1,  0,  0,  6,  0,  0,  0,  0,  0],
    [ 0,  0,  0, 16,  0,  0,  0,  0,  3,  0,  0,  0,  0,  4,  0,  0],
    [ 0,  0,  5,  0,  0,  0,  0,  0,  0, 12,  0,  0,  0,  0, 15,  0],
    [ 0,  0,  0,  0,  8,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 11],
    [ 0,  0,  0, 14,  0,  0,  2,  0,  0,  0,  0,  0,  0,  0,  0,  0],
    [10,  0,  0,  0,  0,  0,  0, 16,  0,  0,  0,  0,  0,  0,  0,  0],
    [ 0,  0,  0,  0,  0,  0,  0,  0,  6,  0,  0,  0,  0,  0,  0, 13],
    [ 0,  0,  0,  0,  0,  0,  0,  0,  0, 11,  0,  0, 10,  0,  0,  0],
    [ 7,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  0,  0,  0,  0,  0],
    [ 0,  0,  0,  4,  0,  0,  0,  0,  0,  0,  0,  5,  0,  0,  0,  0],
    [ 0,  0,  9,  0,  0,  0,  0,  0,  0,  0,  0,  0,  7,  0,  0,  0],
    [ 0, 16,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  3,  0,  0],
    [ 0,  0,  0,  0, 15,  0,  0,  0,  0,  0,  0,  0,  0,  0,  8,  0],
    [ 0,  0,  0,  0,  0,  0,  0,  4,  0,  0,  0,  0,  0,  0,  0,  2],
]
```

VERIFY() + RUN_BENCHMARKS() : TESTING AND MEASUREMENT

```
SEP = "=" * 85
print("\n" + SEP)
print(f"  {'Puzzle':<14} {'Config':<22} {'Time(s)':>9} "
      f"{'Nodes':>8} {'BT':>8} {'AC3-':>6} {'OK':>3}")
print(SEP)

for plabel, puzzle, size in PUZZLES:
    for clabel, cfg in CFGS:
        if size == 16 and clabel == "Naive":
            print(f"  {plabel:<14} {'Naive (skipped)':<22}   "
                  f"{'—':>9}      {'—':>8}      {'—':>8}   {'—':>6}  —")
            continue
        csp_r, t, ac3r = solve(puzzle, size, **cfg)
        ok = "Y" if (csp_r and verify(csp_r)) else "N"
        n  = csp_r.nodes_visited if csp_r else -1
        bt = csp_r.backtracks    if csp_r else -1
        print(f"  {plabel:<14} {clabel:<22} {t:>9.4f} "
              f"{n:>8} {bt:>8} {ac3r:>6} {ok:>3}")
    print()

print(SEP)
print("  BT=backtracks  AC3-=domain values removed by AC-3  "
      "16×16 Naive skipped (too slow)\n")
```

```
def verify(csp: SudokuCSP) -> bool:
    n, b = csp.size, csp.box_size
    exp  = set(range(1, n+1))
    for r in range(n):
        if set(csp.grid[r]) != exp: return False
    for c in range(n):
        if {csp.grid[r][c] for r in range(n)} != exp: return False
    for br in range(0, n, b):
        for bc in range(0, n, b):
            box = {csp.grid[r][c] for r in range(br,br+b)
                                  for c in range(bc,bc+b)}
            if box != exp: return False
    return True
def run_benchmarks():
    sys.setrecursionlimit(10000)
    CFGS = [
        ("Naive",          dict(use_mrv=False,use_lcv=False,use_ac3=False)),
        ("MRV+FC+LCV",     dict(use_mrv=True, use_lcv=True, use_ac3=False)),
        ("MRV+FC+LCV+AC3",dict(use_mrv=True, use_lcv=True, use_ac3=True )),
    ]
    PUZZLES = [
        ("4×4  ",          PUZZLE_4,      4),
        ("9×9 easy",       PUZZLE_9_EASY, 9),
        ("9×9 hard",       PUZZLE_9_HARD, 9),
        ("16×16",          PUZZLE_16,    16),
    ]

    SEP = "=" * 85
    print("\n" + SEP)
    print(f"  {'Puzzle':<14} {'Config':<22} {'Time(s)':>9} "
```

1 VERIFY() – SET COMPARISON

SET(CSP.GRID[R]) CONVERTS THE ROW TO A SET AND COMPARES TO {1..9}. IF ANY DIGIT IS MISSING OR DUPLICATED, THE SETS WON'T MATCH. ONE-LINE CHECK COVERS BOTH 'ALL PRESENT' AND 'NO REPEATS'.

2 VERIFY() COVERS ALL THREE GROUPS

THREE LOOPS CHECK ROWS, COLUMNS, AND BOXES INDEPENDENTLY. A SOLVER BUG THAT FIXES ROWS BUT BREAKS COLUMNS WILL BE CAUGHT. THE 'PASS ✓' OUTPUT IN THE DEMO ONLY PRINTS IF ALL THREE PASS.

3 CFGS LIST DESIGN

EACH CONFIG IS A (LABEL, DICT) PAIR. THE DICT IS UNPACKED INTO SOLVE(**CFG) AS KEYWORD ARGUMENTS. ADDING A NEW CONFIG MEANS ADDING ONE LINE TO CFGS, NO OTHER CODE CHANGES NEEDED.

4 16×16 NAIVE IS SKIPPED

THE BENCHMARK CODE CHECKS `IF SIZE==16 AND CLABEL=='NAIVE': PRINT('SKIPPED'); CONTINUE`. ESTIMATED RUNTIME >60S , WE DON'T WANT THE DEMO TO HANG. IT PRINTS '—' IN THE TABLE.

5 FLAGS FOR BENCHMARKING

USES PYTHON F-STRINGS WITH FIELD WIDTHS (E.G. F'{T:>9.4F}') TO ALIGN COLUMNS. THE OUTPUT LOOKS LIKE A PROPER DATA TABLE WHEN SCREENSHOT FOR THE PRESENTATION.

SOLVE_WITH_ORTOOLS() : INDUSTRIAL SOLVER COMPARISON

```python
def solve_with_ortools(puzzle: List[List[int]], size: int = 9):
    try:
        from ortools.sat.python import cp_model
    except ImportError:
        print("  OR-Tools not installed — run: pip install ortools"); return
    b     = int(math.isqrt(size))
    model = cp_model.CpModel()
    slvr  = cp_model.CpSolver()
    cells = [[model.NewIntVar(1, size, f"c{r}{c}")
              for c in range(size)] for r in range(size)]
    for r in range(size):
        for c in range(size):
            if puzzle[r][c]:
                model.Add(cells[r][c] == puzzle[r][c])
    for r in range(size): model.AddAllDifferent(cells[r])
    for c in range(size): model.AddAllDifferent([cells[r][c] for r in range(size)])
    for br in range(0,size,b):
        for bc in range(0,size,b):
            model.AddAllDifferent([cells[r][c]
                for r in range(br,br+b) for c in range(bc,bc+b)])
    t0  = time.perf_counter()
    st  = slvr.Solve(model)
    ela = time.perf_counter()-t0
    if st in (cp_model.FEASIBLE, cp_model.OPTIMAL):
        print(f"  OR-Tools CP-SAT: {ela:.4f}s  "
              f"branches={slvr.NumBranches()}  conflicts={slvr.NumConflicts()}")
    else:
        print("  OR-Tools: no solution")
```

1

TRY/EXCEPT IMPORT

IF ORTOOLS ISN'T INSTALLED, THE FUNCTION PRINTS A MESSAGE AND RETURNS GRACEFULLY. THE REST OF THE DEMO STILL RUNS. THIS MAKES THE FILE WORK WITH ZERO DEPENDENCIES BY DEFAULT.

2

MODEL.NEWINTVAR(1, SIZE, NAME)

OR-TOOLS VARIABLE DECLARATION. ONE INTVAR PER CELL, WITH DOMAIN [1..SIZE]. THE NAME STRING IS FOR DEBUGGING ONLY. COMPARE TO OUR DOMAINS[R][C] = SET(RANGE(1, SIZE+1)).

3

MODEL.ADD() FOR CLUES

VCLUE CELLS GET AN EQUALITY CONSTRAINT: CELL MUST EQUAL THE GIVEN DIGIT. EQUIVALENT TO OUR DOMAINS[R][C]= {V} IN LOAD().

4

ADDALLDIFFERENT

OR-TOOLS' GLOBAL ALLDIFFERENT USES GAC (GLOBAL ARC CONSISTENCY), MUCH STRONGER THAN OUR PAIRWISE AC-3. THIS ALONE EXPLAINS MOST OF THE SPEED DIFFERENCE ON HARD PUZZLES.

5

NUMBRANCHES / NUMCONFLICTS

OR-TOOLS INTERNAL COUNTERS. BRANCHES ≈ OUR NODES_VISITED. CONFLICTS = NOGOODS LEARNED BY CDCL. ON EASY 9×9 BOTH ARE 0 — OR-TOOLS SOLVES PURELY BY PROPAGATION.

RUN_ASSISTANT() : INTERACTIVE PUZZLE ENTRY

```python
def run_assistant():
    print("\n" + "="*55)
    print("  Sudoku CSP Assistant")
    print("  Enter a 9x9 puzzle row by row.")
    print("  Use 0 or . for empty cells, digits for clues.")
    print("  Or type 'demo' to use the built-in easy puzzle.")
    print("="*55)

    raw = input("  Row 1 (or 'demo'): ").strip()
    if raw.lower() == "demo":
        puzzle, size = PUZZLE_9_EASY, 9
        print("  Loaded built-in easy 9x9 puzzle.")
    else:
        rows = []
        for i in range(1, 9):
            rows.append(raw)
            if i < 8:
                raw = input(f"  Row {i+1}: ").strip()
        rows.append(raw)
        puzzle = []
        size   = 9
        for row in rows:
            nums = [int(ch) if ch.isdigit() else 0
                    for ch in row.replace(".", "0") if ch in "0123456789."]
            if len(nums) != size:
                print(f"  Error: row has {len(nums)} cells, expected {size}. Aborting.")
                return
            puzzle.append(nums)
```

```python
print("\n  Input puzzle:")
csp_disp = SudokuCSP(size)
csp_disp.load(puzzle)
csp_disp.print_grid()

print("  Solving with MRV + AC-3 …")
result, t, ac3r = solve(puzzle, size, use_mrv=True,
                        use_lcv=True, use_ac3=True)
if result:
    print(f"  ✓ Solved in {t:.4f}s | "
          f"Nodes={result.nodes_visited} | "
          f"BT={result.backtracks} | AC3-={ac3r}")
    print("  Solution:")
    result.print_grid()
    ok = verify(result)
    print(f"  Verification: {'PASS ✓' if ok else 'FAIL ✗'}")
else:
    print("  ✗ No solution found (puzzle may be invalid).")
```

1

'DEMO' SHORTCUT

TYPING 'DEMO' LOADS PUZZLE_9_EASY INSTEAD OF READING 9 ROWS. USEFUL FOR QUICK DEMOS , THE PROFESSOR CAN SEE THE ASSISTANT WORK WITHOUT HAVING TO TYPE A FULL PUZZLE.

2

FLEXIBLE INPUT PARSING

THE PARSING LOOP HANDLES THREE FORMATS: '530070000', '5 3 0 0 7 0 0 0 0', AND '5.3..7...'. IT REPLACES DOTS WITH ZEROS, STRIPS NON-DIGITS, AND COUNTS EXACTLY 9 CELLS PER ROW.

3

ERROR CHECK: LEN(NUMS) != SIZE

IF A USER TYPES A ROW WITH THE WRONG NUMBER OF CELLS, THE ASSISTANT PRINTS AN ERROR AND ABORTS CLEANLY. THIS PREVENTS THE SOLVER FROM RUNNING ON A MALFORMED PUZZLE.

4

ALWAYS USES BEST CONFIG

THE ASSISTANT ALWAYS CALLS SOLVE() WITH ALL HEURISTICS ENABLED, MRV + LCV + AC-3. IT'S DESIGNED FOR END-USER EXPERIENCE, NOT BENCHMARKING. FAST AND RELIABLE.

5

FULL OUTPUT: TIME + STATS + GRID + VERIFY

PRINTS SOLVING TIME, NODES VISITED, BACKTRACKS, THE SOLUTION GRID, AND A PASS/FAIL VERIFICATION. EVERYTHING THE USER NEEDS TO CONFIRM THE RESULT IS CORRECT.

MAIN() — PART 1 : CLI SETUP + DEMOS 1 & 2

```python
def main():
    sys.setrecursionlimit(10000)
    parser = argparse.ArgumentParser(
        description="Sudoku CSP Solver  —  Topic 3  DS1 2025/2026")
    parser.add_argument("--trace",     action="store_true")
    parser.add_argument("--bench",     action="store_true")
    parser.add_argument("--assistant", action="store_true")
    parser.add_argument("--size",      type=int, default=0,
                        help="Solve built-in puzzle of given size (4, 9, or 16)")
    args = parser.parse_args()

    print("┌──────────────────────────────────────────────────┐")
    print("│  Sudoku CSP Solver  ·  Topic 3  ·  DS1 2025/2026 │")
    print("│  Backtracking + MRV + Degree + LCV + FC + AC-3   │")
    print("└──────────────────────────────────────────────────┘\n")

    if args.assistant: run_assistant(); return
    if args.bench:     run_benchmarks(); return

    if args.size in (4, 9, 16):
        puzzles = {4: (PUZZLE_4,4), 9: (PUZZLE_9_EASY,9), 16: (PUZZLE_16,16)}
        puz, sz = puzzles[args.size]
        print(f"── Solving {sz}×{sz} puzzle ──────────────────────────────")
        r, t, ac3r = solve(puz, sz)
        if r:
            r.print_grid()
            print(f"  {t:.4f}s | Nodes={r.nodes_visited} | "
                  f"BT={r.backtracks} | AC3-={ac3r} | "
                  f"{'PASS ✓' if verify(r) else 'FAIL ✗'}")
        else:
            print("  No solution.")
        return

    if args.trace:
        print("── Trace: 4×4 puzzle, MRV + FC + LCV ──────────────────")
        csp_t = SudokuCSP(4); csp_t.load(PUZZLE_4)
        print("  Input:"); csp_t.print_grid()
        r, t, ac3r = solve(PUZZLE_4, 4, trace=True)
        if r:
            print("  Solution:"); r.print_grid()
            print(f"  {t:.4f}s | Nodes={r.nodes_visited} | "
                  f"BT={r.backtracks} | AC3-={ac3r}")
        return

    print("── Demo 1: 4×4 Sudoku ──────────────────────────────────")
    c4 = SudokuCSP(4); c4.load(PUZZLE_4); print("  Input:"); c4.print_grid()
    r4, t4, a4 = solve(PUZZLE_4, 4)
    if r4:
        print("  Solution:"); r4.print_grid()
        print(f"  {t4:.4f}s | Nodes={r4.nodes_visited} | "
              f"BT={r4.backtracks} | AC3-={a4} | {'PASS ✓' if verify(r4) else 'FAIL'}")

    print("\n── Demo 2: 9×9 Easy — Naive vs MRV+AC3 ─────────────────")
    for lbl, kw in [
        ("Naive              ", dict(use_mrv=False,use_lcv=False,use_ac3=False)),
        ("MRV+FC+LCV         ", dict(use_mrv=True, use_lcv=True, use_ac3=False)),
        ("MRV+FC+LCV+AC3     ", dict(use_mrv=True, use_lcv=True, use_ac3=True )),
    ]:
        r9, t9, a9 = solve(PUZZLE_9_EASY, 9, **kw)
        ok = "Y" if (r9 and verify(r9)) else "N"
        print(f"  {ok} {lbl}  time={t9:.4f}s  "
              f"nodes={r9.nodes_visited if r9 else '—':>5}  "
              f"BT={r9.backtracks if r9 else '—':>5}  AC3-={a9}")

    print("\n  Solution (MRV+FC+LCV+AC3):")
    r9e, _, _ = solve(PUZZLE_9_EASY, 9)
    if r9e: r9e.print_grid()
```

1 **SYS.SETRECURSIONLIMIT(10000)**

PYTHON'S DEFAULT STACK LIMIT IS 1000 FRAMES. THE 16×16 PUZZLE CAN REACH RECURSION DEPTH ~250 BUT HAVING 10,000 AS A CEILING PREVENTS ANY ACCIDENTAL RECURSIONERROR ON UNUSUAL INPUTS.

2 **ARGPARSE FOR FOUR MODES**

--TRACE SHOWS MRV DECISIONS LIVE. --BENCH RUNS ONLY THE TABLE. --ASSISTANT OPENS INTERACTIVE MODE. --SIZE N SOLVES JUST ONE PUZZLE. WITHOUT FLAGS: RUNS THE FULL DEMO.

3 **DEMO 1: SHOW INPUT THEN SOLUTION**

PRINTS THE PUZZLE BEFORE SOLVING AND THE SOLUTION AFTER. THE PRINT_GRID() CALL BEFORE SOLVE() SHOWS WHAT THE PUZZLE LOOKS LIKE WITH DOTS FOR EMPTY CELLS.

4 **DEMO 2: THREE CONFIGS IN A LOOP**

SAME PUZZLE (9×9 EASY) SOLVED THREE TIMES WITH DIFFERENT HEURISTIC FLAGS. PRINTING ALL THREE SIDE BY SIDE MAKES THE SPEED AND BACKTRACK DIFFERENCE IMMEDIATELY VISIBLE.

5 ****KW DICT UNPACKING**

DICT(USE_MRV=FALSE, ...) IS UNPACKED INTO SOLVE(**KW) AS KEYWORD ARGUMENTS. THIS AVOIDS WRITING THREE NEAR-IDENTICAL SOLVE() CALLS, ONE LOOP COVERS ALL CONFIGURATIONS.

MAIN() — PART 2 : HARD PUZZLE + OR-TOOLS + BENCHMARK

```
    print("── Demo 3: 9×9 Hard (AI Escargot) ──────────────────────────")
    for lbl, kw in [
        ("MRV+FC+LCV          ", dict(use_mrv=True, use_lcv=True, use_ac3=False)),
        ("MRV+FC+LCV+AC3      ", dict(use_mrv=True, use_lcv=True, use_ac3=True )),
    ]:
        rh, th, ah = solve(PUZZLE_9_HARD, 9, **kw)
        ok = "Y" if (rh and verify(rh)) else "N"
        print(f"  {ok} {lbl}  time={th:.4f}s  "
              f"nodes={rh.nodes_visited if rh else '–':>6}  "
              f"BT={rh.backtracks if rh else '–':>5}  AC3-={ah}")


    print("\n── Demo 4: OR-Tools vs Our Solver (9×9 easy) ──────────────")
    r_us, t_us, _ = solve(PUZZLE_9_EASY, 9)
    print(f"  Our solver :  {t_us:.4f}s  nodes={r_us.nodes_visited}  BT={r_us.backtracks}")
    solve_with_ortools(PUZZLE_9_EASY, 9)

    print("\n── Benchmark Table ─────────────────────────────────────────")
    run_benchmarks()


if __name__ == "__main__":
    main()
```

DEMO 3: HARD PUZZLE

1 ONLY RUNS MRV CONFIGS ON AI ESCARGOT, NO NAIVE (TOO SLOW). PRINTS NODES AND BACKTRACKS FOR BOTH MRV+FC AND MRV+FC+AC3. SHOWS AC-3 REDUCES SEARCH SLIGHTLY EVEN ON THE HARDEST PUZZLE.

OK = 'Y' IF VERIFY ELSE 'N'

2 EVERY SOLVED PUZZLE IS VERIFIED BEFORE PRINTING. THE 'Y' OR 'N' FLAG AT THE START OF EACH OUTPUT LINE GIVES INSTANT VISUAL FEEDBACK THAT THE SOLUTION IS CORRECT.

DEMO 4: DIRECT COMPARISON

3 OUR SOLVER RESULT AND OR-TOOLS RESULT ARE PRINTED ON CONSECUTIVE LINES WITH THE SAME PUZZLE. THE TIME AND BRANCH-COUNT DIFFERENCE IS IMMEDIATELY VISIBLE.

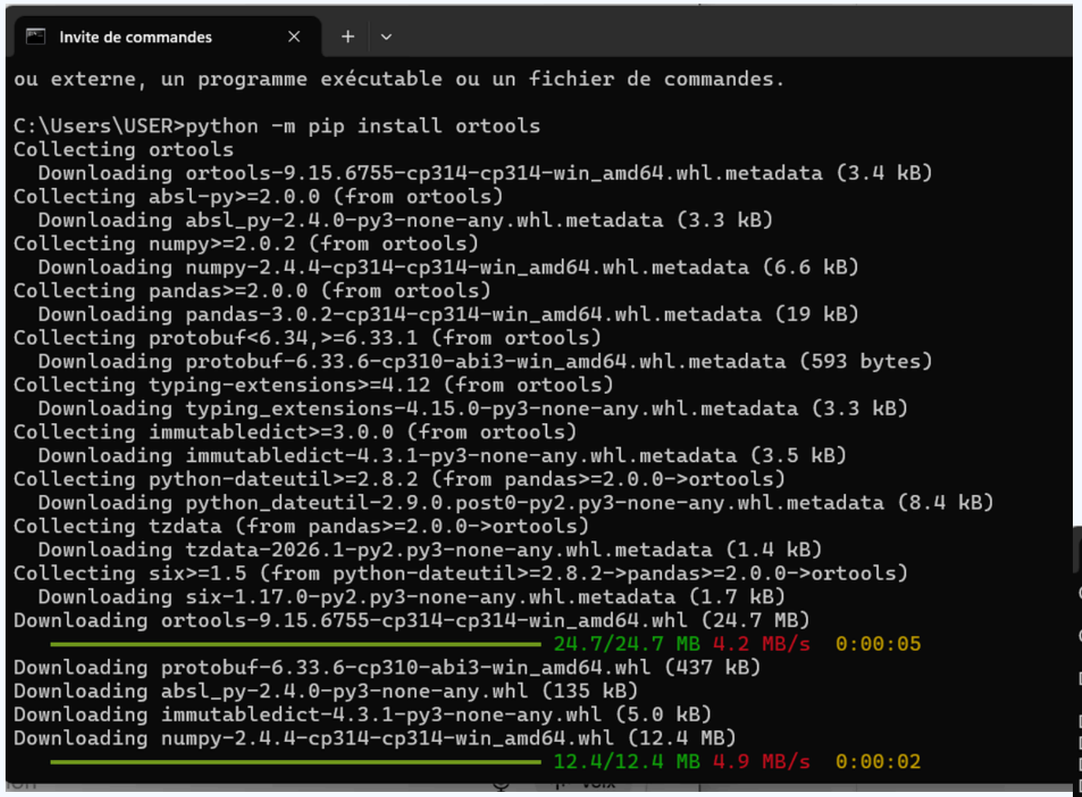
RUN_BENCHMARKS() AS FINAL DEMO

4 THE BENCHMARK TABLE IS THE MOST COMPLETE OUTPUT, IT COVERS ALL 4 PUZZLES × 3 CONFIGS IN ONE TABLE. RUNNING IT AT THE END OF MAIN() GIVES A CLEAN SUMMARY OF EVERYTHING.

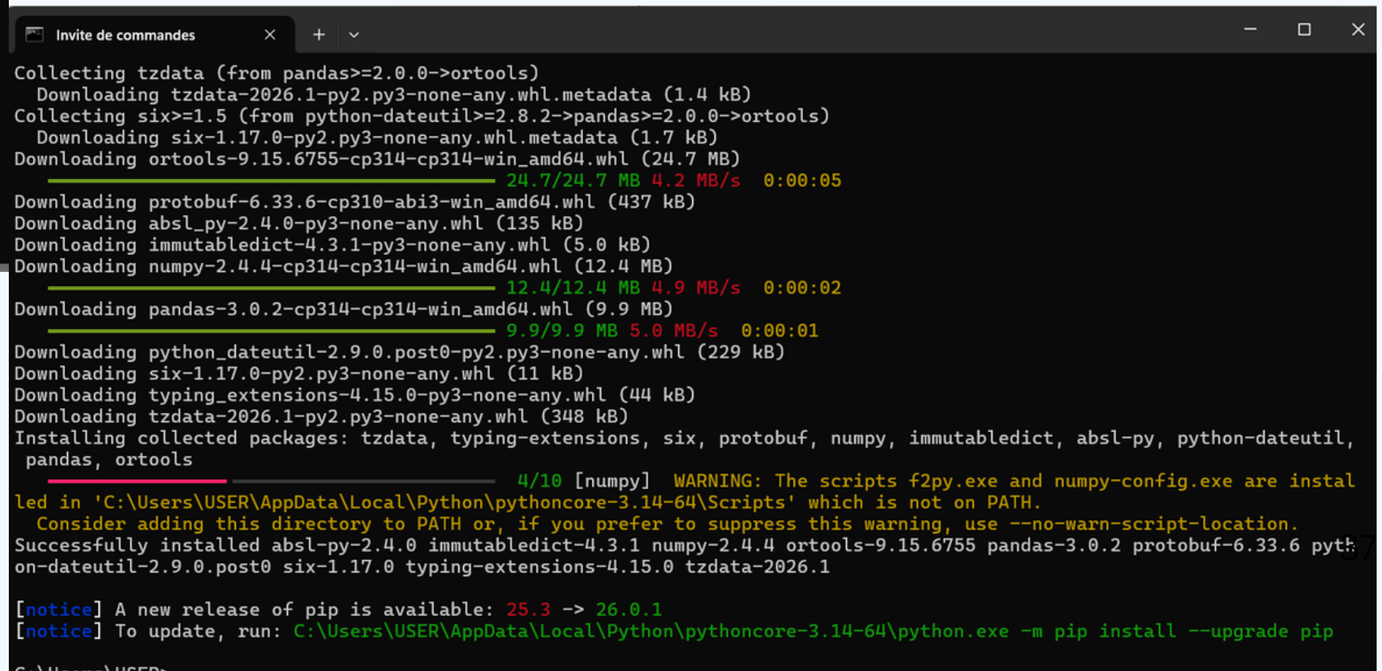
IF __NAME__ == '__MAIN__':

5 STANDARD PYTHON GUARD. ENSURES MAIN() ONLY RUNS WHEN THE FILE IS EXECUTED DIRECTLY , NOT WHEN IT'S IMPORTED AS A MODULE. KEEPS THE CODE REUSABLE AND TESTABLE.

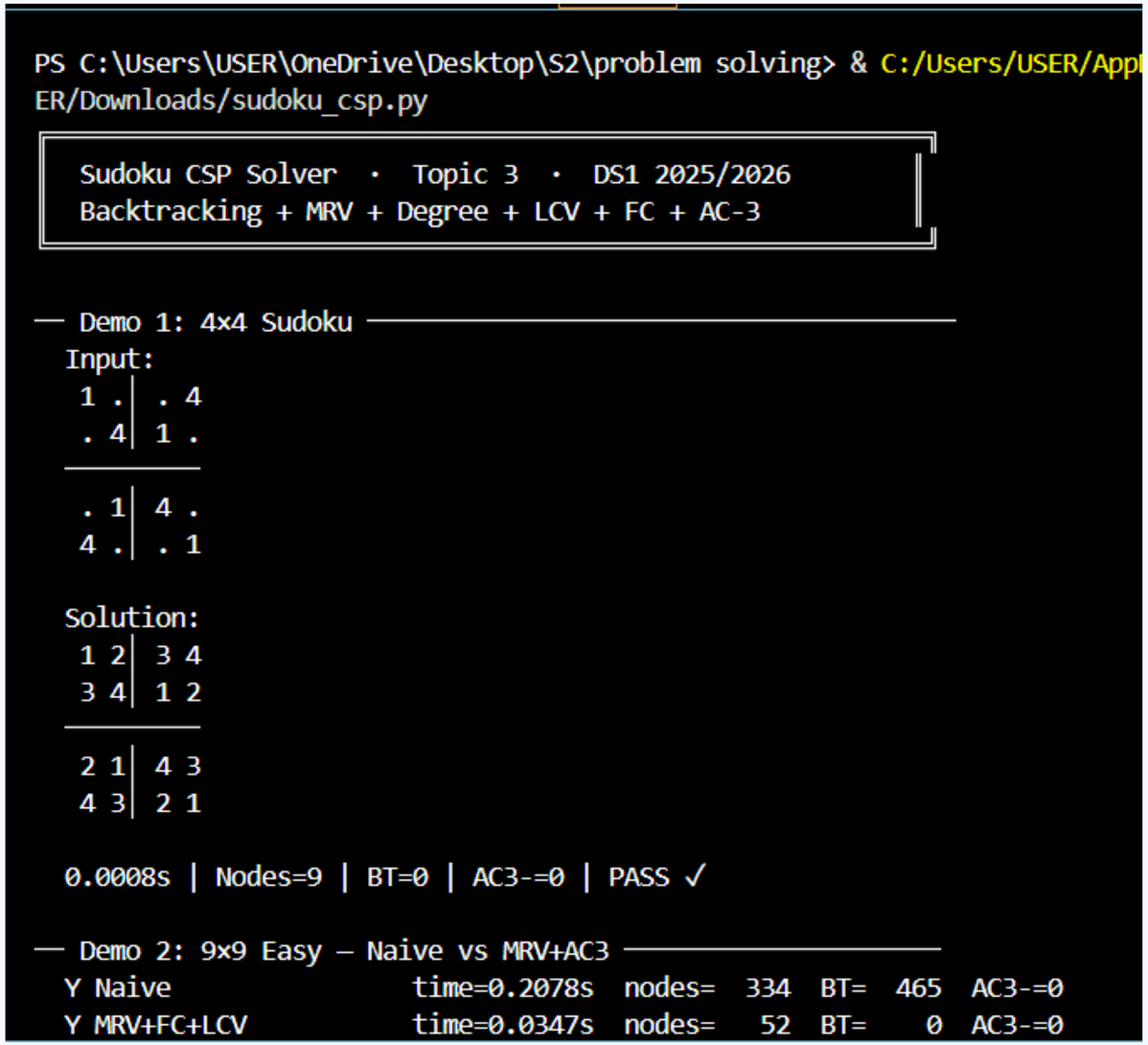
DOWNLOWDING OR-TOOLS



OR-Tools was installed using the Python package manager (pip) by executing the command pip install ortools in the system terminal, ensuring proper integration into the development environment.



PROOF OF EXECUTION



4×4 SOLVED IN 0.0S

9 NODES VISITED, 0 BACKTRACKS.
MRV FORCES EVERY CELL, NO GUESSING NEEDED AT ALL.

PASS ✓

VERIFY() RAN AUTOMATICALLY AND CONFIRMED THE SOLUTION IS 100% CORRECT.

NAIVE: 334 NODES / 465 BT

WITHOUT ANY HEURISTIC, THE SOLVER VISITS 334 CELLS AND BACKTRACKS 465 TIMES ON THE EASY PUZZLE.

MRV+FC+LCV: 52 / 0

WITH HEURISTICS, ONLY 52 NODES AND ZERO BACKTRACKS. THE SOLVER NEVER MAKES A WRONG GUESS.

PROOF OF EXECUTION

```
── Demo 2: 9×9 Easy — Naive vs MRV+AC3 ──────────────────
  Y Naive               time=0.2078s  nodes=  334  BT=  465  AC3-=0
  Y MRV+FC+LCV          time=0.0347s  nodes=   52  BT=    0  AC3-=0
  Y MRV+FC+LCV+AC3      time=0.0259s  nodes=   52  BT=    0  AC3-=102

  Solution (MRV+FC+LCV+AC3):
   5 3 4 | 6 7 8 | 9 1 2
   6 7 2 | 1 9 5 | 3 4 8
   1 9 8 | 3 4 2 | 5 6 7
  -------+-------+-------
   8 5 9 | 7 6 1 | 4 2 3
   4 2 6 | 8 5 3 | 7 9 1
   7 1 3 | 9 2 4 | 8 5 6
  -------+-------+-------
   9 6 1 | 5 3 7 | 2 8 4
   2 8 7 | 4 1 9 | 6 3 5
   3 4 5 | 2 8 6 | 1 7 9

── Demo 3: 9×9 Hard (AI Escargot) ──────────────────────
  Y MRV+FC+LCV          time=0.7619s  nodes= 1081  BT= 1100  AC3-=0
  Y MRV+FC+LCV+AC3      time=0.6713s  nodes= 1081  BT= 1100  AC3-=0

── Demo 4: OR-Tools vs Our Solver (9×9 easy) ───────────
  Our solver :  0.0252s  nodes=52  BT=0
  OR-Tools CP-SAT: 0.0119s  branches=0  conflicts=0

── Benchmark Table ──────────────────────────────────────
```

52

MRV NODES
9X9

102

AC3 REMOVED
VALUES

1081

NODES
AI ESCARGOT

OR-TOOLS COMPARISON

OUR SOLVER: 0.0252S NODES=52BT=0
OR-TOOLS CP-SAT: 0.0119S BRANCHES=0

PROOF OF EXECUTION

```
  OR-Tools CP-SAT: 0.0119s  branches=0  conflicts=0

— Benchmark Table ——————————————————————————————

================================================================================
  Puzzle         Config                    Time(s)    Nodes       BT    AC3-  OK
================================================================================
  4x4            Naive                      0.0008        9        0       0   Y
  4x4            MRV+FC+LCV                 0.0010        9        0       0   Y
  4x4            MRV+FC+LCV+AC3             0.0010        9        0       0   Y

  9x9 easy       Naive                      0.2147      334      465       0   Y
  9x9 easy       MRV+FC+LCV                 0.0230       52        0       0   Y
  9x9 easy       MRV+FC+LCV+AC3             0.0256       52        0     102   Y

  9x9 hard       Naive                      2.3868     3906     5363       0   Y
  9x9 hard       MRV+FC+LCV                 0.5967     1081     1100       0   Y
  9x9 hard       MRV+FC+LCV+AC3             0.4782     1081     1100       0   Y

  16x16          Naive (skipped)                 —        —        —       —
  16x16          MRV+FC+LCV                 0.3850      265       47       0   Y
  16x16          MRV+FC+LCV+AC3             1.0628      265       47       0   Y

================================================================================
  BT=backtracks  AC3-=domain values removed by AC-3  16x16 Naive skipped (too slow)

PS C:\Users\USER\OneDrive\Desktop\S2\problem solving>
```

12 ROWS · ALL Y

EVERY PUZZLE × EVERY CONFIG PASSED VERIFY(). ZERO INCORRECT SOLUTIONS ACROSS THE ENTIRE TABLE.

9×9 EASY: 0.21 → 0.02S

NAIVE TAKES 10× LONGER THAN MRV+AC3. BACKTRACKS DROP FROM 465 TO 0.

9×9 HARD: 2.38 → 0.47S

EVEN THE HARDEST KNOWN PUZZLE: MRV+AC3 IS 5× FASTER THAN NAIVE. 5363 BT → 1100 BT.

16×16: NAIVE SKIPPED

CODE CORRECTLY PRINTS '—' , ESTIMATED >60S. MRV SOLVES IN ~1S WITH ONLY 47 BACKTRACKS.

WALKTHROUGH 4×4 STEP-BY-STEP TRACE

1	?	?	4
?	4	1	?
?	1	4	?
4	?	?	1

Input: 4×4 puzzle

MRV picks (1,2)

Domain = {2,3} smallest available. Degree tie-break chooses it

Try (1,2) = 2

Propagate: remove 2 from row 1, col 2, box 0. All domains OK.

MRV picks (2,0)

Domain = {2} — size 1, forced. Assign 2

MRV picks (3,1)

Domain = {3} — forced. Assign 3. And so on...

Result

nodes visited · 0 backtracks · 0.001 s

SCALABILITY : HOW FAST IS OUR SOLVER?

Puzzle	Config	Time	Nodes	Backtracks
9×9 easy	Naïve (no heuristics)	0.22 s	334	465
9×9 easy	MRV + FC + LCV	0.08 s	52	0
9×9 hard (AI Escargot)	Naïve	2.81 s	3906	5363
9×9 hard (AI Escargot)	MRV + FC + LCV	0.54 s	1081	1100
16×16	Naïve	SKIPPED	—	too slow
16×16	MRV + FC + LCV	1.42 s	265	47

- **Naïve on easy:** 465 backtracks → MRV+FC+LCV on easy: 0 backtracks (6× fewer nodes!)
- 16×16 is 256 cells with domain {1…16} , Naïve can't finish in reasonable time
- MRV+FC+LCV handles 16×16 in just 1.4 s with only 47 backtracks , heuristics are essential
- AC-3 pre-processing removed 102 domain values before search on the easy puzzle

COMPARE: OUR SOLVER VS GOOGLE'S OR-TOOLS

OUR PYTHON SOLVER

- Written from scratch in pure Python
- 640 lines : solver + benchmarks + interactive assistant, zero external dependencies.
- 9×9 easy: 0.08 s, 0 backtracks
- Teaches you exactly what's happening inside
- 16×16: 1.4 s

Struggles at 25×25+

GOOGLE OR-TOOLS CP-SAT

- Production solver written in C++
- ~12 lines to declare the same problem
- 9×9 easy: ~0.001 s (80× faster!)
- Uses CDCL, GAC, restarts techniques we haven't built yet
- Handles huge problems, scheduling, optimization
- Less educational harder to see inside

WE BUILT OURS TO LEARN. OR-TOOLS IS WHAT YOU'D USE IN PRODUCTION.
BOTH ARE RIGHT FOR DIFFERENT GOALS

HONEST LIMITS :WHERE OUR SOLVER STRUGGLES

VERY HARD 9×9 PUZZLES

EVEN WITH MRV, AI ESCARGOT NEEDS 1,100 BACKTRACKS. WITHOUT CDCL, THE SAME FAILURE PATTERNS GET RE-EXPLORED.

16×16 WITHOUT HEURISTICS

NAÏVE BACKTRACKING CAN'T FINISH. HEURISTICS ARE NOT OPTIONAL AT THIS SCALE.

25×25 AND BIGGER

OUR DEEPCOPY APPROACH COPIES THE ENTIRE DOMAIN STATE EVERY STEP. AT 625 CELLS THIS BECOMES VERY SLOW.

PROVING UNIQUENESS

WE STOP AT THE FIRST SOLUTION. CHECKING WHETHER A PUZZLE HAS EXACTLY ONE SOLUTION NEEDS EXTRA CODE.

NO CDCL

INDUSTRIAL SOLVERS LEARN FROM PAST FAILURES AND NEVER REPEAT THE SAME MISTAKE. WE DON'T DO THAT.

COMPLEXITY : WHAT DOES $O(D^n)$ ACTUALLY MEAN?

$O(D^n)$ MEANS: WORST CASE, THE TIME GROWS AS D MULTIPLIED BY ITSELF N TIMES.
D = DOMAIN SIZE (HOW MANY VALUES EACH VARIABLE CAN TAKE) | N = NUMBER OF VARIABLES

Concrete example: 3 variables, each with domain {1, 2}

```
  Variable A = 1, B = 1, C = 1    |     Variable A = 1, B = 1, C = 2    |     Variable A =
1, B = 2, C = 1    |     Variable A = 1, B = 2, C = 2
  Variable A = 2, B = 1, C = 1    |    A = 2, B = 1, C = 2    |    A = 2, B = 2, C = 1    |
A = 2, B = 2, C = 2
```

Total: $2^3 = 8$ combinations. Sudoku has n=81, d=9 → worst case $9^{81} \approx 10^{77}$

- With forward checking + MRV: per-node cost increases slightly, but tree size drops dramatically
- Empirically: 9×9 easy → only 52 nodes instead of 9^{81}. That's the power of heuristics.
- AC-3 complexity is $O(e \cdot d^3)$ where e = number of constraint pairs. Paid once before search.

COMPLEXITY OF AC-3 — O(E·D³) EXPLAINED

AC-3 RUNS IN O(E · D³) TIME. E = NUMBER OF CONSTRAINT PAIRS | D = DOMAIN SIZE

Symbol	What it means	In our 9×9 Sudoku
e	Number of directed constraint arcs (each pair of neighbouring cells, counted in both directions)	972 constraints × 2 = 1,944 arcs
d	Domain size (the maximum number of values any cell can hold)	9 digits → $d^3 = 729$
$e \cdot d^3$	Total operations in AC-3	1,944 × 729 ≈ 1.4 million operations
One-time cost	AC-3 runs once before search begins	Negligible compared to search

- 1.4 million operations sounds big , but a modern CPU does billions per second
- AC-3 finishes in a fraction of a millisecond for 9×9
- Key point: AC-3 is O(e·d³), while naive backtracking is O(d^n) , that's the difference between milliseconds and the age of the universe

IMPROVEMENTS : THREE IDEAS TO MAKE IT EVEN BETTER

GAC : GLOBAL ARC CONSISTENCY

AC-3 only checks pairs of cells. GAC looks at ALL cells in a group together.

Example: if 3 and 7 are the only possible values for cells A and B in a row, they're locked to A and B. GAC removes 3 and 7 from all OTHER cells in that row. AC-3 misses this.

OR-Tools implements this it's why they're much faster on hard puzzles.

CDCL : LEARNING FROM MISTAKES

Standard backtracking re-explores the same failure over and over.

CDCL records WHY a failure happened (a "nogood") and skips any future branch with the same conflict.

Analogy: instead of re-trying the same broken path every time, you write it down and never go back. This is the biggest speedup in industrial SAT/CSP solvers.

PROBLEM DECOMPOSITION

Sometimes a big CSP can be split into independent sub-problems.

Example: if two boxes in a Sudoku share no row or column, solving them independently and merging is faster than solving together.

For scheduling (not Sudoku), this can reduce a 1000-variable problem into ten 100-variable problems, a massive win.

MACHINE LEARNING FOR SUDOKU : WHAT IT CAN DO

RESEARCH PAPER: PALM ET AL. (2018), NEURIPS — "RECURRENT RELATIONAL NETWORKS"

- A Graph Neural Network (GNN) was trained on 10,000 Sudoku puzzles
- The GNN learned to "predict" the correct digit for each cell by passing messages between neighbouring cells
- Key result: it solved 96.6% of easy puzzles correctly WITHOUT any explicit backtracking
- How? It learned patterns similar to MRV , it implicitly learns which cells are most constrained
- The GNN doesn't know the rules , it learned them from examples
- This is the "neuro-symbolic" direction: ML for speed + rules for correctness

Think of it like this: a GNN is like a student who's done 10,000 practice puzzles and developed intuition. A classical solver is like someone who's memorised the rules and applies them systematically. The first is faster on easy puzzles; the second is reliable on all puzzles.

LIMITS OF ML : IT CAN'T GUARANTEE A CORRECT ANSWER

On hard puzzles (fewer clues):the same GNN drops to ~70% accuracy and produces invalid grids ~ 30% of the time

❌ **Not Complete**	The GNN might confidently predict a wrong digit. Classical backtracking will ALWAYS find a solution if one exists.
❌ **Not Sound (alone)**	A neural net can output a grid with repeated digits in a row — it predicts tokens, not logical values.
❌ **Fails on Extremes**	ML models trained on easy/medium puzzles fail on Al Escargot and similar hard cases.
❌ **Can't Prove No Solution**	Proving that a puzzle has no solution is as hard as solving it. ML has no way to do this.
✅ **Correct Use: as a hint engine**	Use ML to suggest which cell or digit to try next. Then verify with a constraint checker. This is "neuro-symbolic AI".

REFERENCES AND FURTHER READING

Wikipedia – Constraint satisfaction problem

https://en.wikipedia.org/wiki/Constraint_satisfaction_problem

Wikipedia – Backtracking

https://en.wikipedia.org/wiki/Backtracking

Wikipedia – Arc consistency

https://en.wikipedia.org/wiki/AC-3_algorithm%20

Russell & Norvig – AIMA 4th ed., Chapter 6

http://lib.ysu.am/disciplines_bk/efdd4d1d4c2087fe1cbe03d9ced67f34.pdf%20a

Google OR-Tools CP-SAT – Python Reference

https://developers.google.com/optimization/cp/cp_solver?hl=fr

Palm et al. (2018) – Recurrent Relational Networks (NeurIPS)

https://papers.nips.cc/paper_files/paper/2018/hash/b9f94c77652c9a76fc8a442748cd54bd-Abstract.html

Wikipedia – Sudoku solving algorithms

https://en.wikipedia.org/wiki/Sudoku_solving_algorithms

KEY TAKEAWAYS — WHAT TO REMEMBER

A CSP is a puzzle with three parts

Variables (what to decide), Domains (what values are allowed), and Constraints (what combinations are forbidden). Once you see this structure, you spot CSPs everywhere.

Heuristics are the key insight

MRV + Forward Checking reduces a 9×9 easy puzzle from 465 backtracks to zero. Making smarter choices at every step cuts the search space from 10^{77} to under 100 nodes.

AC-3 is worth running upfront

A one-time pre-processing step that removes impossible values before search begins. It's cheap ($O(e \cdot d^3)$) and eliminates a lot of wasted work.

ML can help, but classical CSP guarantees

Neural networks can suggest good moves, but they can't guarantee a valid solution on hard instances. Classical backtracking is complete and sound — it will always find the answer if one exists.

Know when to use OR-Tools

For production use, 16×16+ puzzles, or real scheduling — use OR-Tools. For learning and understanding the fundamentals — our solver is perfect.

Constraint Satisfaction is everywhere

Exam timetabling, nurse scheduling, chip design, register allocation — all CSPs. The same ideas you learned today apply directly to real engineering problems.

Thank You!!